MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

LAB PROJECT REPORT

For

ECE2244: VLSI Design Lab

"DESIGN AND IMPLEMENTATION OF 32-BIT CENTRAL PROCESSING UNIT USING CADENCE 90nm TECHNOLOGY"

Submitted by

Nyasha Singh

Hari R

Shashank Mallya

Reg No- 240959002

240959004

240959006

Batch: A1

DEPT.OF ELECTRONICS AND COMMUNICATION (ECE)

MANIPAL INSTITUTE OF TECHNOLOGY, MANIPAL-576104

INDEX

Sl. No.	CONTENT	Pg. No.
1.	OBJECTIVE	1
2.	INTRODUCTION	2-4
3.	PROJECT THEORY AND EXPLANATION	5-10
4.	VERILOG	11-21
5.	REPORTS	22-26
6.	CONCLUSION	27
7.	REFERENCES	28

1. OBJECTIVE

The objective of this project is to design and implement a 32-bit CPU architecture using Verilog HDL. The processor integrates essential components such as a Register File, Arithmetic Logic Unit (ALU), Control Unit (CU), RAM (Data Memory) and Instruction Memory to execute instructions efficiently.

The goal is to simulate a working processor capable of performing arithmetic, logical operations and data transfer between memory and registers.

2. INTRODUCTION

A Central Processing Unit (CPU) is the heart of a computer system. It is responsible for executing instructions and controlling all operations in the system. Every task performed by a computer, whether simple or complex, is handled by the CPU.

In this project, a simplified 32-bit CPU is designed using Verilog HDL. A 32-bit CPU means that it can process 32-bit data at a time, which allows faster and more efficient computation.

The CPU is made up of several important components:

- Instruction Memory
- Register File
- ALU (Arithmetic Logic Unit)
- Control Unit
- Data Memory

Each of these components performs a specific function, and together they ensure that the CPU works correctly.

1. Instruction Memory:

Instruction Memory stores all the instructions that the CPU needs to execute. Each instruction is stored at a specific address. The CPU uses the Program Counter (PC) to fetch instructions one by one from this memory.

This memory is usually read-only during execution, meaning instructions are not changed while the program is running. It plays a key role in the fetch stage of the CPU.

2. Register File:

The Register File is a collection of small, high-speed storage locations called registers. These registers are used to temporarily store data during execution.

The register file allows:

- Reading data from registers
- Writing results back to registers

Registers are much faster than memory, so they are used for intermediate calculations and quick data access.

3. ALU (Arithmetic Logic Unit):

The ALU is responsible for performing all arithmetic and logical operations in the CPU.

Some common operations include:

- Addition and subtraction
- AND, OR, XOR operations
- Comparison operations

The ALU takes input from registers and performs operations based on control signals. It is one of the most important parts of the CPU because it actually performs the computation.

4. Control Unit:

The Control Unit acts like the “brain” of the CPU. It controls how all other components work together.

The Control Unit:

- Decodes the instruction
- Generates control signals
- Decides what operation needs to be performed

For example, it tells:

- The ALU what operation to do
- The register file when to read/write
- The memory when to access data

Without the control unit, the CPU cannot function properly.

5. Data Memory (RAM):

Data Memory is used to store data values during program execution. Unlike instruction memory, this memory can be both read and written.

It is mainly used in:

- Load operations (reading data)
- Store operations (writing data)

This allows the CPU to handle large amounts of data beyond registers.

3. PROJECT THEORY AND EXPLANATION – 32 BIT CPU

3.1 THEORY

Central Processing Unit, or CPU, is the most important and functional unit of a computer. Also called “the brain of computers,” CPUs take in data, executes the user-defined instructions, and show the final, processed output. It can also store information, run the computer programs, and manage the functioning of the computer. CPUs are placed in sockets on the computer’s motherboard, connecting it to all the computer parts.

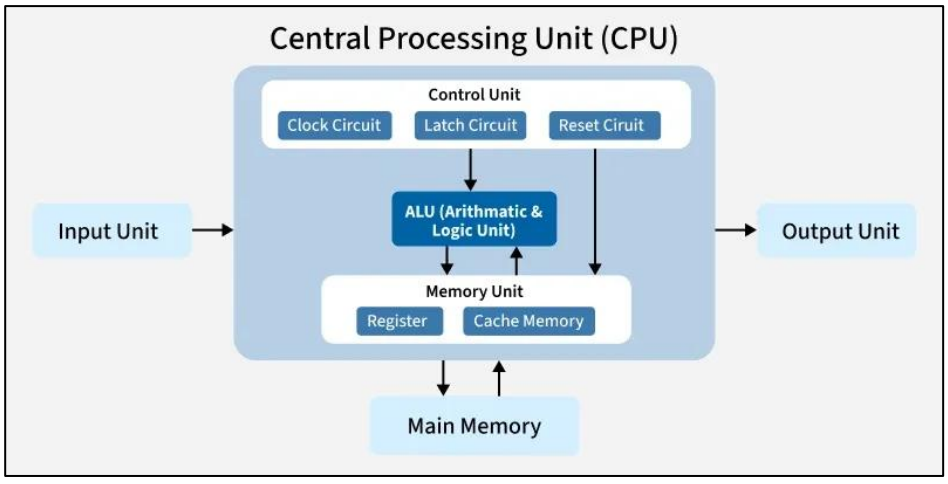
CPU was made possible after the invention of the transistor in 1947 by John Bardeen, Walter Brattain, and William Shockley. Jack Kilby and Robert Noyce created the integrated circuit (IC) in 1958. This made it possible to combine many transistors into a single chip. The first microprocessor was made by Intel in 1971, and since then, companies like Intel, AMD and Motorola have made faster, smaller yet powerful CPUs with many features and functionalities.

A CPU consists of two types of memory: Random Access Memory (RAM) and Read Only Memory (ROM). RAM is the primary storage responsible for storing temporary data that is ready to use whenever needed. ROM stores data permanently in secondary storage devices like hard drive. It can only be accessed but cannot be altered. The CPU is made up of many components that work together in its functioning:

- Control Unit (CU): Guides the system, synchronizes components’ work,

assigns and executes instructions. It performs based on the speed of the clock.

- Arithmetic Logic Unit (ALU): Handles the arithmetic and logical operations. It contains two or more registers, through which the user's input is taken and generates the result in the accumulator.
- Memory Management Unit: Handles memory usage and data flow between RAM and CPU.
- Cache: High-speed, small storage component that handles routine tasks. It is located on the processor chip which eliminates waiting time. The CPU only retrieves data from RAM if it is not loaded in cache.
- Registers: Permanent memories that fetches constant and immediate data for smooth functioning.
- Clock: Coordinates the components to work synchronously. The clock speed determines the rate of instructions handled by CPU.
- Instruction Register: Contains the next instruction to be executed.
- Instruction Pointer: Specifies location of the next instruction.
- Buses: Ensures proper dataflow within the components.



(Image Source: https://media.geeksforgeeks.org/wp-content/uploads/20250719153951741828/CPU-Components-.webp)

CPUs functions based on CPU instruction cycle, managed by control unit at the clock's pace. The cycle consists of the fetch-decode-execute process. Instructions are fetched from memory and placed on register. It is then decoded by ALU and executed based on the user's commands. Executed results are shown in the accumulator and are stored in memory.

A 32-bit CPU refers to the size of data it can handle in a single operation. A register in this CPU has 32 individual bits. Thus, it can store:

- From 0 to 2^{32} -1 unsigned integers,
- From -2^{31} to 2^{31}-1 signed integers.

CPU performs ALU operations of 32-bit numbers in one cycle and can transfer 32 bits data between CPU and memory per operation.

Memory addresses are also 32 bits wide; hence the maximum addressable memory is 2^{32} = 4,294,967,296 addresses, that is, 4GB of RAM.

Technological advancements have enhanced CPUs to become faster and more efficient, while reducing the size and power. Microprocessors utilize CPUs miniaturized into a small integrated circuit chip. Each microprocessor can have cores, which are the smallest physical hardware unit capable of performing processing tasks. These could be single-core, or dual, or quad, or multi-core. Each subsequent integration of these "mini-CPUs" can give higher and faster CPU performance, better multitasking and handling heavy tasks like video editing and gaming. GPUs (Graphic Processing Units) are another integration that can enhance graphics and image performance.

3.2 PROJECT EXPLANATION

All modules work together to execute instructions in a sequential fetch, decode, execute, and write-back cycle. The process begins with the Program Counter (PC) generating the address of the current instruction. This address is used to fetch the instruction from instruction memory. The Control Unit then decodes this instruction and produces control signals to determine the operation type, data source, and destination. The Register File reads the required operands based on the instruction fields, while an immediate value is generated and selected when needed. These inputs go to the ALU, which performs arithmetic or logical operations such as addition, subtraction, AND, OR, multiplication, or comparison, depending on the control signals. The ALU output is either used as the result or as an address for data memory in load/store operations, where data can be read or written. A multiplexer then selects between the ALU result and memory output, and the final result is written back into the Register File if enabled. Meanwhile, the PC updates to the next address, ensuring continuous instruction execution. Thus, the entire file represents a fully functional CPU data path and control system that works together to process instructions step by step.

VERILOG MODULES:

1. PROGRAM COUNTER:

This module acts like the CPU's internal pointer and holds the address of the currently executing instruction. On every rising edge of the clock, it captures the next instruction address (PC+4). When the active-low asynchronous reset signal is high, it forces the address back to 0x00000000 asynchronously. The reset is

asynchronous so that the PC can be initialized regardless of clock state.

2. CONTROL UNIT:

Purely combinational block that decodes the 6-bit opcode field (instruction [31:26]) and generates the necessary control signals to steer data through the data path. Acts as the “brain” of the processor, translates what the instruction requests and fulfils it. It generates control signals such as:

- reg_write
- alu_src
- mem_to_reg
- mem_read
- mem_write
- alu_ctrl

3. REGISTER FILE:

It is a synchronous read/write storage element which contains 32 registers each 32 bits wide in the form of an array block. It allows the CPU to read two source operands (rs1 and rs2) at the same clock cyle and write one result(rd) back on the next clock edge when the write-enable is high. The register r0 is wired to zero, it always returns 32’b0 and write operations targeting r0 are ignored.

4. 32-BIT ALU:

This module consists of arithmetic and logical operations based on the 4bit alu_ctrl signal. It operates on two 32-bit operands and produces a single 32-bit

result. Multiplication is implemented using a 64-bit intermediate product, with only lower 32-bits returned.

5. TOP MODULE:

The top module serves as the main unit that connects all submodules of the 32-bit CPU and manages the instruction execution process. It links the Program Counter, Control Unit, Register File, ALU, and memory interfaces to create a complete data path. The Program Counter produces the instruction address, which fetches instructions from instruction memory. The Control Unit decodes these instructions and generates control signals that direct the Register File and ALU's operations. The Register File provides input operands, and the ALU carries out the necessary computation. Depending on the instruction, data can be read from or written to data memory. Finally, the outcome—either from the ALU or memory—is written back to the Register File. In this way, the top module ensures all components work together to execute instructions in a fetch, decode, execute cycle.

4. VERILOG

4.1 CODE

```
`timescale 1ns / 1ps

//================ PROGRAM COUNTER ================
module pc_reg (
    input  wire        clk,
    input  wire        rst_n,
    input  wire [31:0] next_pc,
    output reg  [31:0] pc
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            pc <= 32'd0;
        else
            pc <= next_pc;
    end
endmodule

//================ CONTROL UNIT ================
module control_unit (
    input  wire [5:0] opcode,
    output reg        reg_write,
    output reg        alu_src,
    output reg        mem_to_reg,
```

```
    output reg       mem_read,
    output reg       mem_write,
    output reg [3:0]  alu_ctrl
);
    always @(*) begin
        reg_write  = 0;
        alu_src    = 0;
        mem_to_reg = 0;
        mem_read   = 0;
        mem_write  = 0;
        alu_ctrl   = 4'b0000;

        case (opcode)
            6'b000000: begin reg_write=1; alu_ctrl=4'b0000; end
            6'b000001: begin reg_write=1; alu_ctrl=4'b0001; end
            6'b000010: begin reg_write=1; alu_ctrl=4'b0010; end
            6'b000011: begin reg_write=1; alu_ctrl=4'b0011; end
            6'b000100: begin reg_write=1; alu_ctrl=4'b0110; end
            6'b000101: begin reg_write=1; alu_src=1; alu_ctrl=4'b0000; end
            6'b000110: begin reg_write=1; alu_src=1; mem_to_reg=1; mem_read=1; end
            6'b000111: begin alu_src=1; mem_write=1; end
            6'b001000: begin reg_write=1; alu_ctrl=4'b0100; end
            6'b001001: begin reg_write=1; alu_ctrl=4'b1001; end
            default: ;
        endcase
    end
endmodule
```

```
//================ REGISTER FILE =================
module reg_file (
    input  wire       clk,
    input  wire       rst_n,
    input  wire       we,
    input  wire [4:0]  rs1,
    input  wire [4:0]  rs2,
    input  wire [4:0]  rd,
    input  wire [31:0] wd,
    output wire [31:0] rd1,
    output wire [31:0] rd2
);
    reg [31:0] regs [0:31];
    integer i;

    assign rd1 = (rs1==0) ? 32'd0 : regs[rs1];
    assign rd2 = (rs2==0) ? 32'd0 : regs[rs2];

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            for (i=0;i<32;i=i+1) regs[i] <= 32'd0;
        else if (we && rd!=0)
            regs[rd] <= wd;
    end
endmodule
```

```
//================ ALU ================
module alu_32 (
    input  wire [31:0] a,
    input  wire [31:0] b,
    input  wire [3:0]  alu_ctrl,
    output reg  [31:0] y
);
    wire [63:0] mul_res = a * b;

    always @(*) begin
        case (alu_ctrl)
            4'b0000: y = a + b;
            4'b0001: y = a - b;
            4'b0010: y = a & b;
            4'b0011: y = a | b;
            4'b0100: y = mul_res[31:0];
            4'b0110: y = a ^ b;
            4'b1001: y = ($signed(a) < $signed(b)) ? 32'd1 : 32'd0;
            default: y = 32'd0;
        endcase
    end
endmodule

//================ TOP MODULE ================
module cpu_core (
    input  wire       clk,
    input  wire       rst_n,
```

```
    input  wire [31:0] imem_rdata,
    input  wire [31:0] dmem_rdata,

    output wire [31:0] imem_addr,
    output wire [31:0] dmem_addr,
    output wire [31:0] dmem_wdata,
    output wire        dmem_we,
    output wire        dmem_re
);

    wire [31:0] pc, instr;
    wire [31:0] r1, r2, alu_out, imm_ext, result;
    wire reg_write, alu_src, mem_to_reg, mem_read, mem_write;
    wire [3:0] alu_ctrl;
    wire [4:0] rd;

    assign instr = imem_rdata;
    assign imem_addr = pc;

    assign imm_ext = {{16{instr[15]}}, instr[15:0]};

    assign rd = (instr[31:26]==6'b000101 || instr[31:26]==6'b000110) ?
                instr[20:16] : instr[15:11];

    pc_reg PC (.clk(clk), .rst_n(rst_n), .next_pc(pc+4), .pc(pc));
```

```
control_unit CU (
    .opcode(instr[31:26]),
    .reg_write(reg_write),
    .alu_src(alu_src),
    .mem_to_reg(mem_to_reg),
    .mem_read(mem_read),
    .mem_write(mem_write),
    .alu_ctrl(alu_ctrl)
);

reg_file RF (
    .clk(clk), .rst_n(rst_n), .we(reg_write),
    .rs1(instr[25:21]), .rs2(instr[20:16]),
    .rd(rd), .wd(result),
    .rd1(r1), .rd2(r2)
);

alu_32 ALU (
    .a(r1),
    .b(alu_src ? imm_ext : r2),
    .alu_ctrl(alu_ctrl),
    .y(alu_out)
);

assign dmem_addr  = alu_out;
assign dmem_wdata = r2;
assign dmem_we    = mem_write;
```

```
    assign dmem_re    = mem_read;

    assign result = mem_to_reg ? dmem_rdata : alu_out;

endmodule
```

4.2 TESTBENCH

```
`timescale 1ns / 1ps

module cpu_tb;

    reg clk, rst_n;

    wire [31:0] imem_addr, dmem_addr, dmem_wdata;
    reg  [31:0] imem_rdata, dmem_rdata;
    wire dmem_we, dmem_re;

    cpu_core DUT (
        .clk(clk), .rst_n(rst_n),
        .imem_rdata(imem_rdata),
        .dmem_rdata(dmem_rdata),
        .imem_addr(imem_addr),
        .dmem_addr(dmem_addr),
        .dmem_wdata(dmem_wdata),
        .dmem_we(dmem_we),
        .dmem_re(dmem_re)
    );

    // Instruction memory
    reg [31:0] imem [0:63];
    always @(*) imem_rdata = imem[imem_addr[7:2]];
```

```
    // Data memory
    reg [31:0] dmem [0:255];
    always @(posedge clk) if (dmem_we) dmem[dmem_addr[7:0]] <= dmem_wdata;
    always @(*) dmem_rdata = dmem_re ? dmem[dmem_addr[7:0]] : 32'd0;

    // Clock
    always #5 clk = ~clk;

    initial begin
        clk = 0;
        rst_n = 0;
        #10 rst_n = 1;

        // Program
        imem[0] = 32'b000101_00000_00001_0000000000001010; // ADDI R1 = 10
        imem[1] = 32'b000101_00000_00010_0000000000010100; // ADDI R2 = 20
        imem[2] = 32'b000000_00001_00010_00011_00000000000; // ADD R3 = R1+R2

        #100;

        $display("R1 = %0d", DUT.RF.regs[1]);
        $display("R2 = %0d", DUT.RF.regs[2]);
        $display("R3 = %0d (EXPECTED 30)", DUT.RF.regs[3]);
        $finish;
    end
endmodule
```

4.3 SDC file

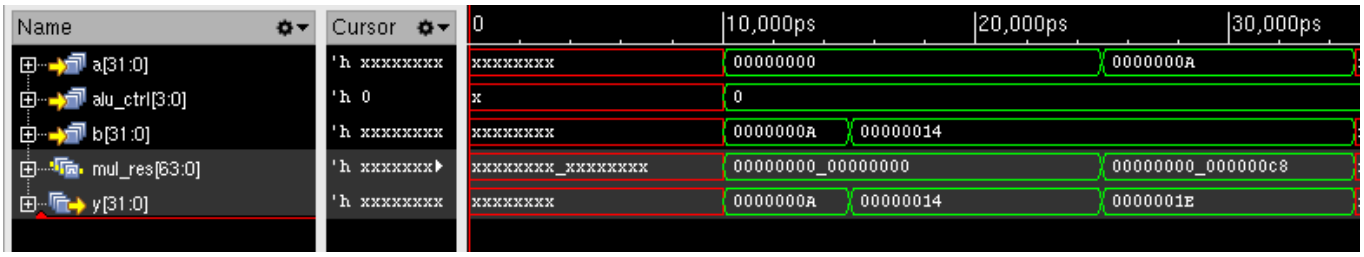
```
# CLOCK
create_clock -name clk -period 10 [get_ports clk]
set_clock_uncertainty 0.2 [get_clocks clk]
# RESET (ASYNC)
# Do not time reset path
set_false_path -from [get_ports rst_n]
# Provide proper drive for reset (removes warning)
set_driving_cell -lib_cell INVX1 [get_ports rst_n]
# INPUT DELAYS (ONLY DATA INPUTS)
set_input_delay 2 -clock clk \
[remove_from_collection [all_inputs] [get_ports {clk rst_n}]]
# OUTPUT DELAYS
set_output_delay 2 -clock clk [all_outputs]
# INPUT DRIVE (DATA INPUTS ONLY)
set_driving_cell -lib_cell INVX1
[remove_from_collection [all_inputs] [get_ports {clk rst_n}]]
# OUTPUT LOAD
set_load 0.1 [all_outputs]
# DESIGN RULE CONSTRAINTS
set_max_transition 0.5 [current_design]
set_max_fanout 10 [current_design]
```

4.4 TCL file

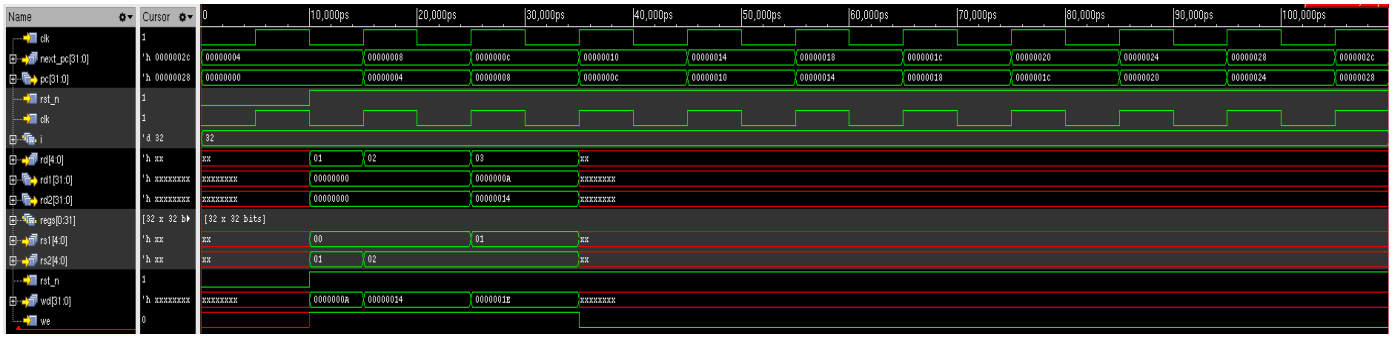
```
read_libs /home/install/FOUNDRY/digital/90nm/dig/lib/slow.lib
read_hdl cpu.v
elaborate
read_sdc cpu.sdc
set_db syn_generic_effort medium
set_db syn_map_effort medium
set_db syn_opt_effort medium
syn_generic
syn_map
syn_opt
report_timing > cpu_timing.rep
report_area > cpu_area.rep
report_power > cpu_pwr.rep
write_hdl > cpu_net.v
write_sdc > cpuop.sdc
gui_show
```

5. REPORTS

5.1 WAVEFORMS



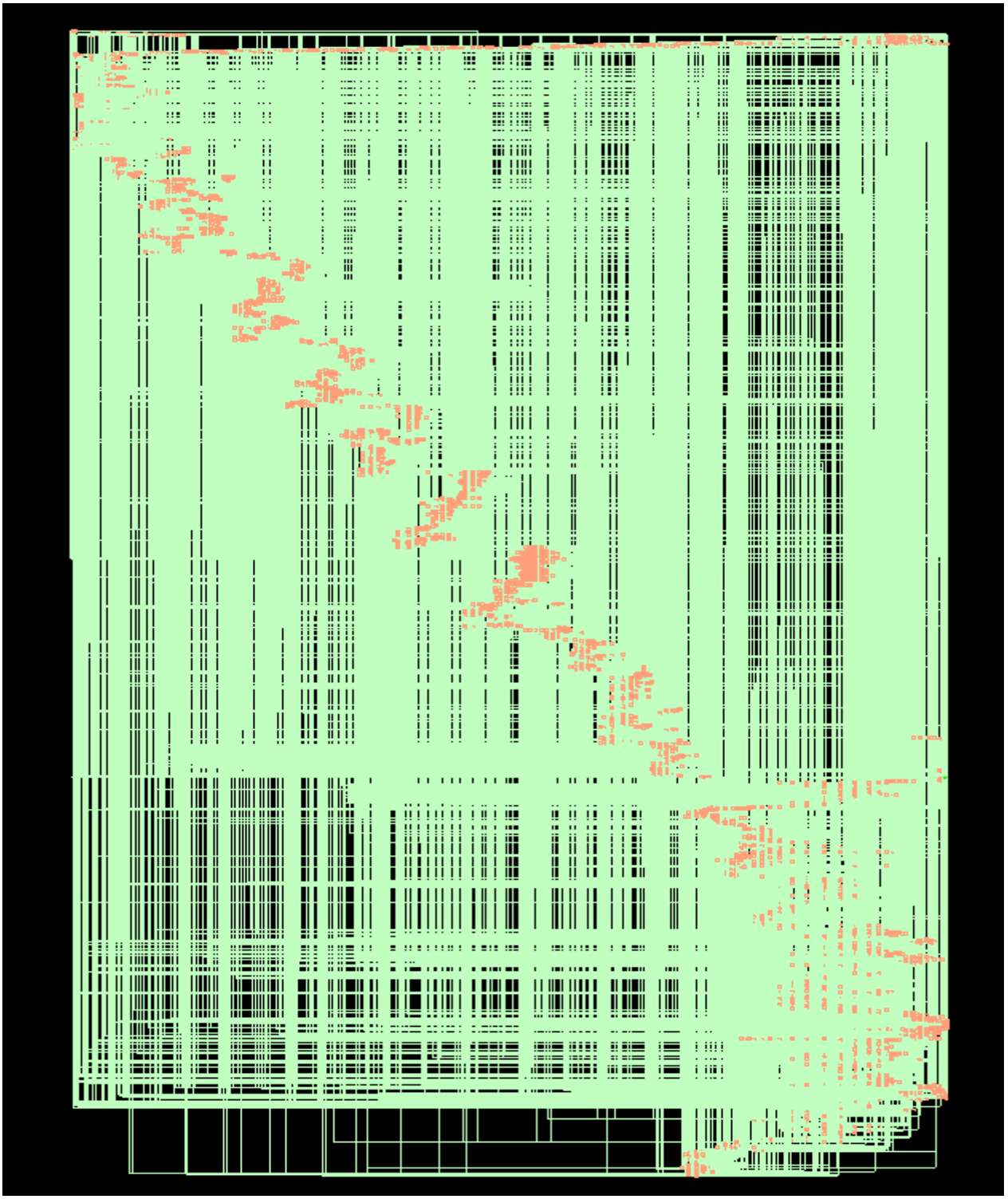
ALU



PC & REG FILE

CPU DUT

5.2 SCHEMATIC



5.3 SYNTHESIS REPORTS

5.3.1 Area Report

Open ▾ cpu_area.rep ~/CPU Save

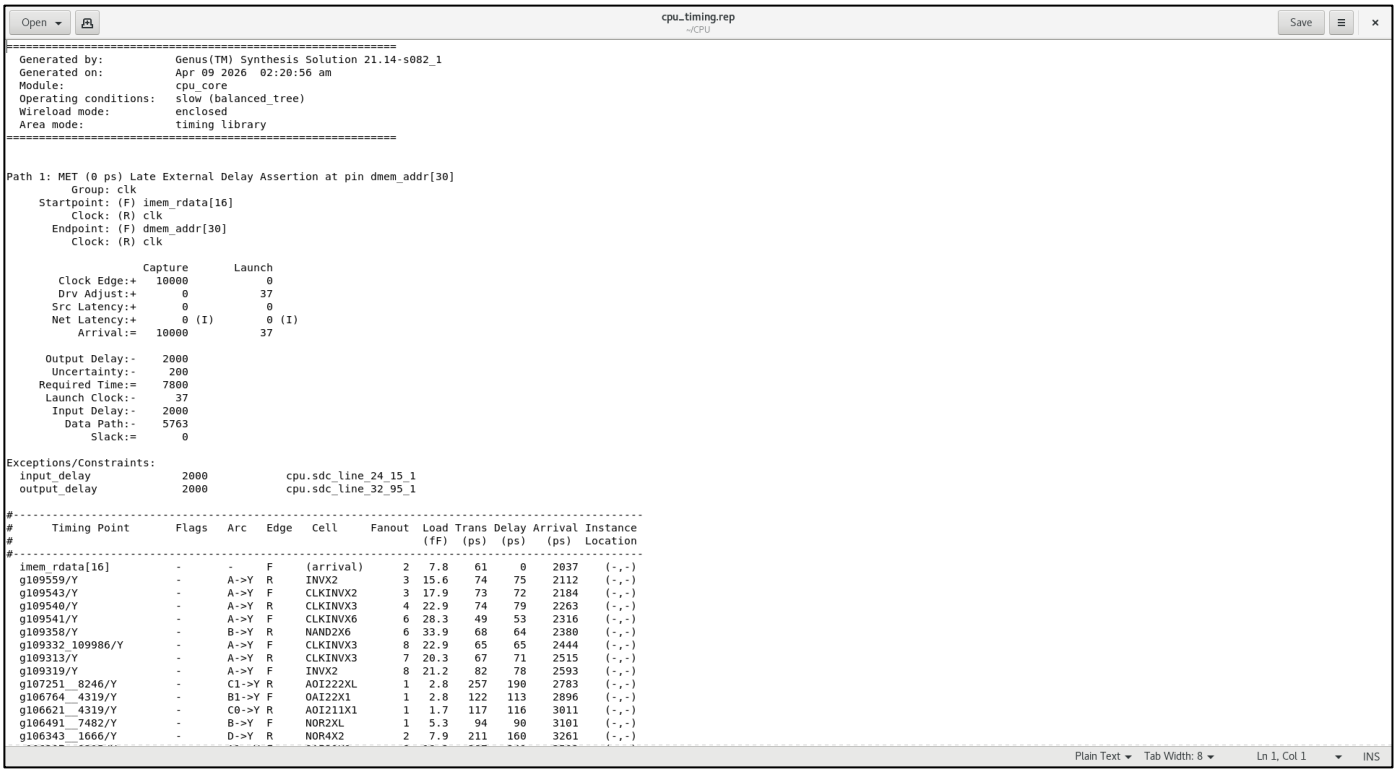
```
============================================================
  Generated by:           Genus(TM) Synthesis Solution 21.14-s082_1
  Generated on:           Apr 09 2026  02:20:56 am
  Module:                 cpu_core
  Operating conditions:   slow (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================

Instance    Module     Cell Count  Cell Area  Net Area   Total Area   Wireload
-----------------------------------------------------------------------------------
cpu_core                     5628  50406.511     0.000    50406.511 <none> (D)
  CU     control_unit          23    100.668     0.000      100.668 <none> (D)
  PC     pc_reg                30    633.525     0.000      633.525 <none> (D)
  (D) = wireload is default in technology library
```

5.3.2 Power Report

Open ▾ cpu_pwr.rep ~/CPU

```
Instance: /cpu_core
Power Unit: W
PDB Frames: /stim#0/frame#0
  ------------------------------------------------------------------------
    Category         Leakage     Internal    Switching        Total    Row%
  ------------------------------------------------------------------------
       memory     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
    register     1.54993e-04  2.69442e-03  5.79821e-05  2.90739e-03  63.79%
       latch     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
       logic     9.84437e-05  8.13314e-04  7.38496e-04  1.65025e-03  36.21%
        bbox     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
       clock     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
         pad     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
          pm     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
  ------------------------------------------------------------------------
    Subtotal     2.53437e-04  3.50773e-03  7.96478e-04  4.55765e-03 100.00%
  Percentage           5.56%       76.96%       17.48%      100.00% 100.00%
  ------------------------------------------------------------------------
```

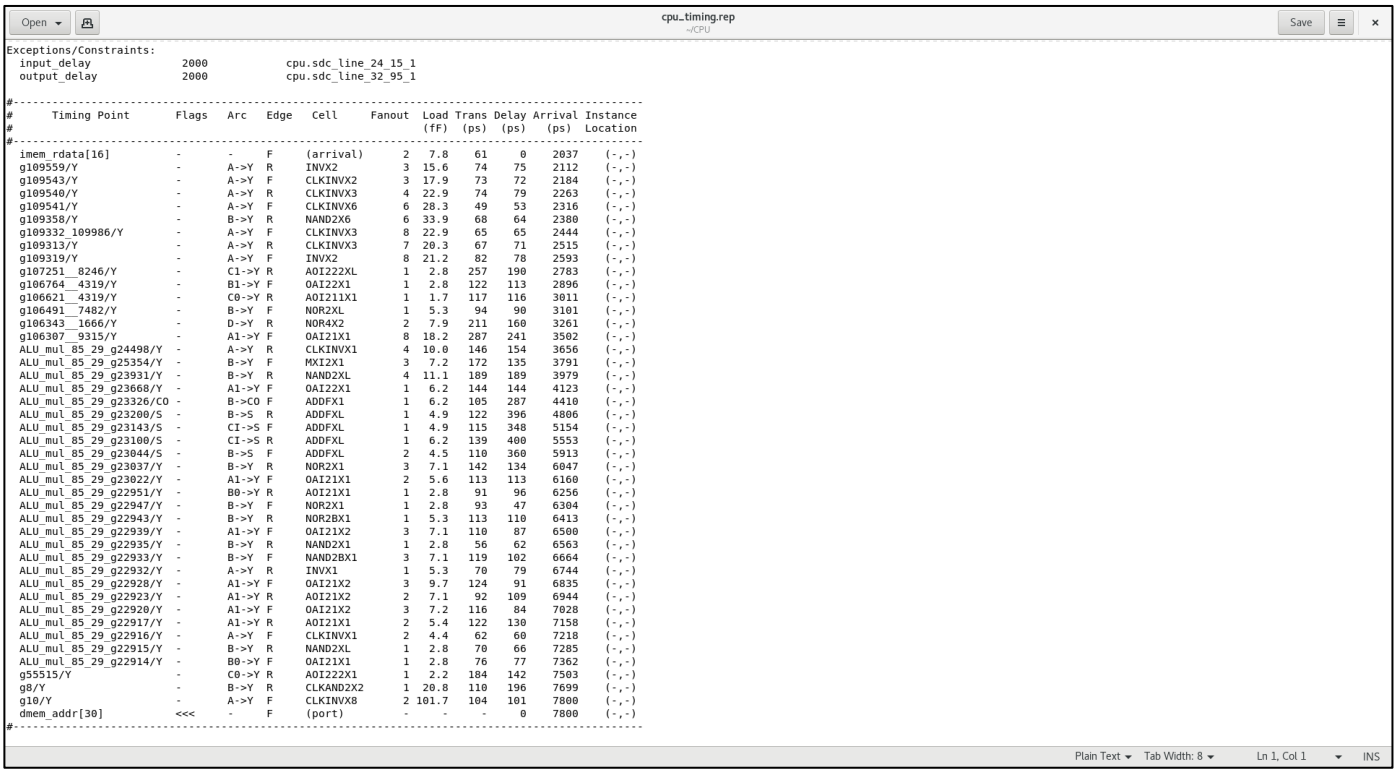
5.3.3 Timing Report



Open | cpu_timing.rep ~/CPU | Save

```
============================================================
  Generated by:           Genus(TM) Synthesis Solution 21.14-s082_1
  Generated on:           Apr 09 2026  02:20:56 am
  Module:                 cpu_core
  Operating conditions:   slow (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================


Path 1: MET (0 ps) Late External Delay Assertion at pin dmem_addr[30]
          Group: clk
     Startpoint: (F) imem_rdata[16]
          Clock: (R) clk
       Endpoint: (F) dmem_addr[30]
          Clock: (R) clk

                    Capture       Launch
       Clock Edge:+   10000            0
       Drv Adjust:+       0           37
      Src Latency:+       0            0
      Net Latency:+       0 (I)        0 (I)
          Arrival:=   10000           37

     Output Delay:-    2000
      Uncertainty:-     200
    Required Time:=    7800
     Launch Clock:-      37
      Input Delay:-    2000
        Data Path:-    5763
            Slack:=       0

Exceptions/Constraints:
  input_delay             2000            cpu.sdc_line_24_15_1
  output_delay            2000            cpu.sdc_line_32_95_1

#-------------------------------------------------------------------------------------------
#      Timing Point       Flags   Arc   Edge   Cell     Fanout  Load Trans Delay Arrival Instance
#                                                          (fF)  (ps)  (ps)   (ps)  Location
#-------------------------------------------------------------------------------------------
  imem_rdata[16]          -       -     F     (arrival)      2   7.8    61     0    2037    (-,-)
  g109559/Y               -       A->Y  R     INVX2          3  15.6    74    75    2112    (-,-)
  g109543/Y               -       A->Y  F     CLKINVX2       3  17.9    73    72    2184    (-,-)
  g109540/Y               -       A->Y  R     CLKINVX3       4  22.9    74    79    2263    (-,-)
  g109541/Y               -       A->Y  F     CLKINVX6       6  28.3    49    53    2316    (-,-)
  g109358/Y               -       B->Y  R     NAND2X6        6  33.9    68    64    2380    (-,-)
  g109332_109986/Y        -       A->Y  F     CLKINVX3       8  22.9    65    65    2444    (-,-)
  g109313/Y               -       A->Y  R     CLKINVX3       7  20.3    67    71    2515    (-,-)
  g109319/Y               -       A->Y  F     INVX2          8  21.2    82    78    2593    (-,-)
  g107251__8246/Y         -       C1->Y R     AOI222XL       1   2.8   257   190    2783    (-,-)
  g106764__4319/Y         -       B1->Y F     OAI22X1        1   2.8   122   113    2896    (-,-)
  g106621__4319/Y         -       C0->Y R     AOI211X1       1   1.7   117   116    3011    (-,-)
  g106491__7482/Y         -       B->Y  F     NOR2XL         1   5.3    94    90    3101    (-,-)
  g106343__1666/Y         -       D->Y  R     NOR4X2         2   7.9   211   160    3261    (-,-)
```

Plain Text | Tab Width: 8 | Ln 1, Col 1 | INS

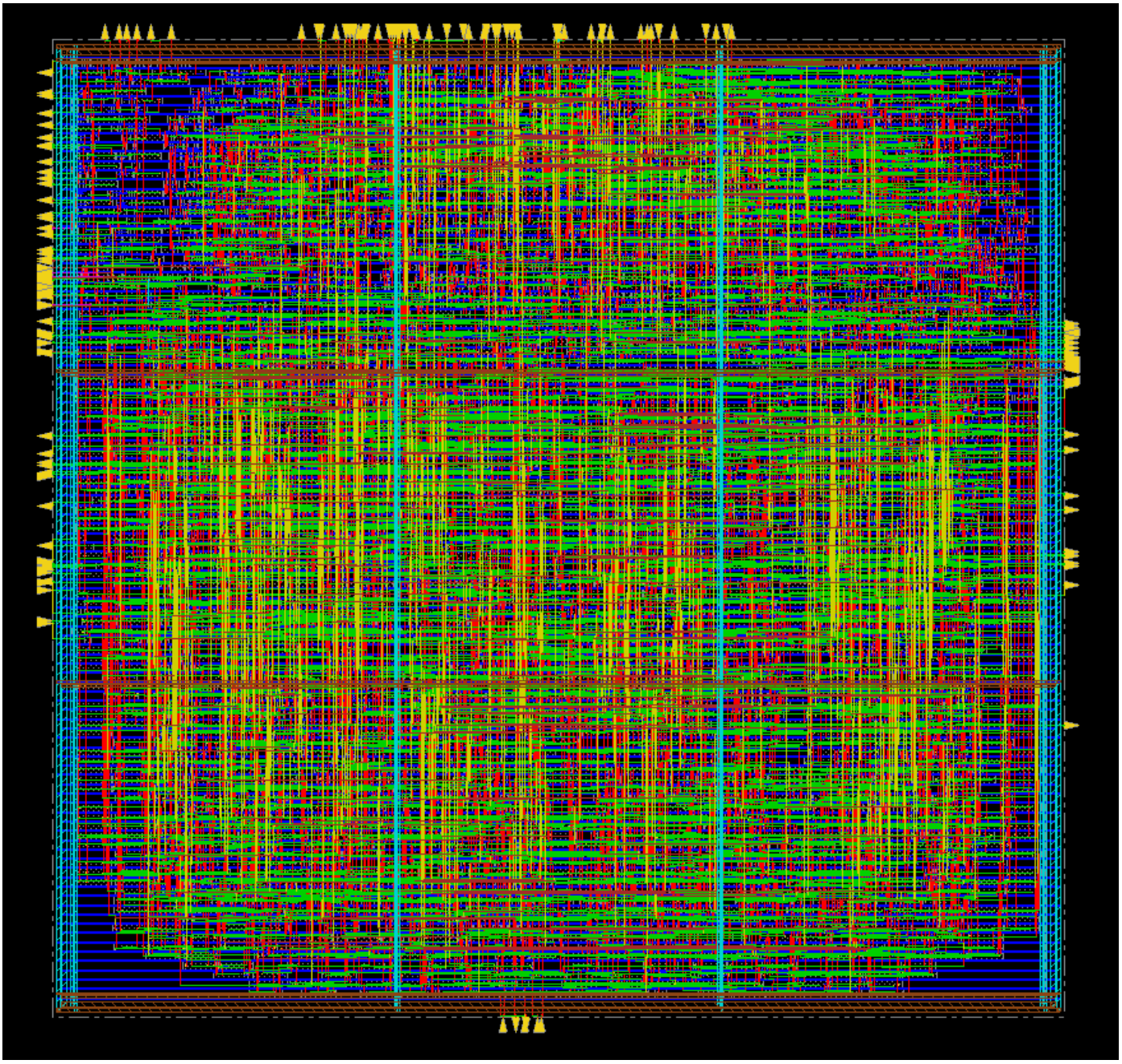


Open | cpu_timing.rep ~/CPU | Save

```
Exceptions/Constraints:
  input_delay             2000            cpu.sdc_line_24_15_1
  output_delay            2000            cpu.sdc_line_32_95_1

#-------------------------------------------------------------------------------------------
#      Timing Point       Flags   Arc   Edge   Cell     Fanout  Load Trans Delay Arrival Instance
#                                                          (fF)  (ps)  (ps)   (ps)  Location
#-------------------------------------------------------------------------------------------
  imem_rdata[16]          -       -     F     (arrival)      2   7.8    61     0    2037    (-,-)
  g109559/Y               -       A->Y  R     INVX2          3  15.6    74    75    2112    (-,-)
  g109543/Y               -       A->Y  F     CLKINVX2       3  17.9    73    72    2184    (-,-)
  g109540/Y               -       A->Y  R     CLKINVX3       4  22.9    74    79    2263    (-,-)
  g109541/Y               -       A->Y  F     CLKINVX6       6  28.3    49    53    2316    (-,-)
  g109358/Y               -       B->Y  R     NAND2X6        6  33.9    68    64    2380    (-,-)
  g109332_109986/Y        -       A->Y  F     CLKINVX3       8  22.9    65    65    2444    (-,-)
  g109313/Y               -       A->Y  R     CLKINVX3       7  20.3    67    71    2515    (-,-)
  g109319/Y               -       A->Y  F     INVX2          8  21.2    82    78    2593    (-,-)
  g107251__8246/Y         -       C1->Y R     AOI222XL       1   2.8   257   190    2783    (-,-)
  g106764__4319/Y         -       B1->Y F     OAI22X1        1   2.8   122   113    2896    (-,-)
  g106621__4319/Y         -       C0->Y R     AOI211X1       1   1.7   117   116    3011    (-,-)
  g106491__7482/Y         -       B->Y  F     NOR2XL         1   5.3    94    90    3101    (-,-)
  g106343__1666/Y         -       D->Y  R     NOR4X2         2   7.9   211   160    3261    (-,-)
  g106307__9315/Y         -       A1->Y F     OAI21X1        8  18.2   287   241    3502    (-,-)
  ALU_mul_85_29_g24498/Y  -       A->Y  R     CLKINVX1       4  10.0   146   154    3656    (-,-)
  ALU_mul_85_29_g25354/Y  -       B->Y  F     MXI2X1         3   7.2   172   135    3791    (-,-)
  ALU_mul_85_29_g23931/Y  -       B->Y  R     NAND2XL        4  11.1   189   189    3979    (-,-)
  ALU_mul_85_29_g23668/Y  -       A1->Y F     OAI22X1        1   6.2   144   144    4123    (-,-)
  ALU_mul_85_29_g23326/CO -       B->CO F     ADDFX1         1   6.2   105   287    4410    (-,-)
  ALU_mul_85_29_g23200/S  -       B->S  R     ADDFXL         1   4.9   122   396    4806    (-,-)
  ALU_mul_85_29_g23143/S  -       CI->S F     ADDFXL         1   4.9   115   348    5154    (-,-)
  ALU_mul_85_29_g23100/S  -       CI->S R     ADDFXL         1   6.2   139   400    5553    (-,-)
  ALU_mul_85_29_g23044/S  -       B->S  F     ADDFXL         2   4.5   110   360    5913    (-,-)
  ALU_mul_85_29_g23037/Y  -       B->Y  R     NOR2X1         3   7.1   142   134    6047    (-,-)
  ALU_mul_85_29_g23022/Y  -       A1->Y F     OAI21X1        2   5.6   113   113    6160    (-,-)
  ALU_mul_85_29_g22951/Y  -       B0->Y R     AOI21X1        1   2.8    91    96    6256    (-,-)
  ALU_mul_85_29_g22947/Y  -       B->Y  F     NOR2X1         1   2.8    93    47    6304    (-,-)
  ALU_mul_85_29_g22943/Y  -       B->Y  R     NOR2BX1        1   5.3   113   110    6413    (-,-)
  ALU_mul_85_29_g22939/Y  -       A1->Y F     OAI21X2        3   7.1   110    87    6500    (-,-)
  ALU_mul_85_29_g22935/Y  -       B->Y  R     NAND2X1        1   2.8    56    62    6563    (-,-)
  ALU_mul_85_29_g22933/Y  -       B->Y  F     NAND2BX1       3   7.1   119   102    6664    (-,-)
  ALU_mul_85_29_g22932/Y  -       A->Y  R     INVX1          1   5.3    70    79    6744    (-,-)
  ALU_mul_85_29_g22928/Y  -       A1->Y F     OAI21X2        3   9.7   124    91    6835    (-,-)
  ALU_mul_85_29_g22923/Y  -       A1->Y R     AOI21X2        2   7.1    92   109    6944    (-,-)
  ALU_mul_85_29_g22920/Y  -       A1->Y F     OAI21X2        3   7.2   116    84    7028    (-,-)
  ALU_mul_85_29_g22917/Y  -       A1->Y R     AOI21X1        2   5.4   122   130    7158    (-,-)
  ALU_mul_85_29_g22916/Y  -       A->Y  F     CLKINVX1       2   4.4    62    60    7218    (-,-)
  ALU_mul_85_29_g22915/Y  -       B->Y  R     NAND2XL        1   2.8    70    66    7285    (-,-)
  ALU_mul_85_29_g22914/Y  -       B0->Y F     OAI21X1        1   2.8    76    77    7362    (-,-)
  g55515/Y                -       C0->Y R     AOI222X1       1   2.2   184   142    7503    (-,-)
  g8/Y                    -       B->Y  R     CLKAND2X2      1  20.8   110   196    7699    (-,-)
  g10/Y                   -       A->Y  F     CLKINVX8       2 101.7   104   101    7800    (-,-)
  dmem_addr[30]           <<<     -     F     (port)         -     -     -     0    7800    (-,-)
#-------------------------------------------------------------------------------------------
```

Plain Text | Tab Width: 8 | Ln 1, Col 1 | INS

5.4 LAYOUT



6. CONCLUSION

The design and implementation of the 32-bit CPU using Verilog successfully show the basic functions of a processor at the hardware level. The project combines key parts like the Program Counter, Control Unit, Register File, ALU, and memory interface into one system that can execute instructions through a structured fetch, decode, execute, and write-back cycle. Each module was designed and tested separately, then merged in the top module to ensure correct data flow and control signal coordination. This project offers a solid understanding of digital design, computer architecture, and hardware description with Verilog. It also lays a good groundwork for future improvements like pipelining, new instruction sets, and performance enhancement.

7. REFERENCES

1. https://www.ibm.com/think/topics/central-processing-unit
2. https://www.lenovo.com/in/en/glossary/what-is-cpu/?orgRef=https%253A%252F%252Fwww.google.com%252F&srsltid=AfmBOopD824KVcDbPE1oWbyxYQgY7hxvIEhLonzvjfnh35mDHqSskTbH
3. https://www.redhat.com/en/blog/cpu-components-functionality
4. https://www.geeksforgeeks.org/computer-science-fundamentals/central-processing-unit-cpu/
5. https://www.arm.com/glossary/cpu
6. https://adacomputerscience.org/concepts/arch_processor
7. https://www.geeksforgeeks.org/computer-science-fundamentals/central-processing-unit-cpu/